GOVERNANCE, PRIVACY & BUSINESS CONSULTING · LESSON 4 OF 5

Data Masking, Row-Level Security and Access Control

Protect sensitive data while keeping it useful. Mask values in queries and in views, show only the last digits of identifiers, restrict which rows each user sees with row-level security, and audit who accessed what, with worked examples in SQL and Python and a regional report protected through RLS.

Gap-fill lesson | ✦ 2026 | Intermediate | ⏱ 120 min

By the end of this lesson you can

- Choose between masking, pseudonymisation and removal for a field
- Mask sensitive values with SQL views and Python functions
- Restrict rows by user with a row-level security rule
- Audit access to sensitive data and review the audit log

Datasets used

`customers.csv` `superstore_sales.csv`

Three ways to protect a field

Method	What it does	Reversible?	Use when
Removal	The field is not stored or not passed on	Not applicable	The purpose does not need it
Masking	Part of the value is hidden (for example, last four digits)	No	People need to recognise a record, not identify the person
Pseudonymisation	The identifier is replaced with a token, kept in a separate key store	Yes, with the key	Analysts need to link records over time, but not see identities

Choose the weakest method that still meets the purpose. Removal is strongest; pseudonymisation keeps the most utility but needs careful key management.

CASE STUDY **Support tickets for a regional review**

A regional manager needs to review support tickets by city and product. The tickets contain customer phone numbers and full names, which the review does not need. The right design removes the names and masks the phone numbers, leaving city, product and issue type. Nothing in the review requires the full values.

Masking in SQL with a view

A **view** is a saved query that presents a table in a safer form. Analysts query the view; the raw table stays restricted to the few people who need it.

Python

```
import pandas as pd
import duckdb

con = duckdb.connect()
customers = pd.read_csv("customers.csv")
con.register("customers_raw", customers)

con.execute("""
    CREATE VIEW customers_masked AS
    SELECT customer_id,
           -- keep the first letter and mask the rest of the name
           LEFT(customer_name, 1) || REPEAT('*', GREATEST(LENGTH(customer_name) - 1, 0)) AS
name_masked,
           segment,
           region
    FROM customers_raw
""")
print(con.execute("SELECT * FROM customers_masked LIMIT 5").fetchdf())
```

OUTPUT

```
  customer_id     name_masked    segment region
0       C0001     S**********  Corporate  North
1       C0002      M*********  Corporate  North
2       C0003       R********   Consumer   East
3       C0004  D**********     Corporate   West
4       C0005     N**********  Corporate  South
```

The view keeps the identifier and the analytical fields, and removes the readable name. A query on the view cannot recover the name, because the function that produced it discards it.

Masking in Python

The same idea works in code. The function below keeps the last four digits of a phone number and hides the rest, which is useful for showing a record to a support agent who needs to confirm identity.

Python

```python
def mask_phone(number: str) -> str:
    digits = "".join(ch for ch in number if ch.isdigit())
    if len(digits) < 4:
        return "*" * len(digits)
    return "*" * (len(digits) - 4) + digits[-4:]

for phone in ["+91 98450 12345", "080-2345-6789", "12"]:
    print(f"{phone:>18} → {mask_phone(phone)}")
```

OUTPUT

```
   +91 98450 12345 → ********2345
     080-2345-6789 → *******6789
                12 → **
```

Masking is only as strong as the rest of the system. If the unmasked column is still readable through another table, a join, or an export, masking has protected nothing.

Pseudonymisation with a key

Pseudonymisation replaces a direct identifier with a token, so that analysts can count distinct customers and follow them across months without knowing who they are. The mapping lives in a separate, restricted table.

Python

```python
import hashlib
import hmac

SECRET_KEY = b"keep-this-in-a-secrets-manager"       # placeholder; never hard-code in production

def pseudonym(customer_id: str) -> str:
    return hmac.new(SECRET_KEY, customer_id.encode(), hashlib.sha256).hexdigest()[:16]

print(pseudonym("C0001"), pseudonym("C0001"), pseudonym("C0002"))
```

OUTPUT

```
2972d88b78de3379 2972d88b78de3379 ddf2a050855650da
```

The same input gives the same token, so counts and joins still work. Using a keyed HMAC rather than a plain hash matters: without the key, someone who knows the possible IDs can hash them and match the token. The key must be stored separately and protected.

::::warn Hashing is not anonymisation A plain hash of a phone number can be reversed by hashing every possible phone number. Use a keyed function, keep the key separate, and treat pseudonymised data as personal data under most privacy laws. ::::

Row-level security

Row-level security (RLS) limits which rows each user can see, within the same table or report. A regional manager sees only their region's rows, and the same report shows different rows to different people without copies of the data.

The rule below is written in the style of a database policy. Check the exact syntax for your platform; the principle is the same.

SQL

```sql
-- PostgreSQL-style row-level security (illustrative)
ALTER TABLE sales ENABLE ROW LEVEL SECURITY;

CREATE POLICY regional_access ON sales
    FOR SELECT
    USING (region = current_setting('app.user_region'));
```

In Power BI, the same idea is built into the data model: a role defines a filter on a table (for example `region = "North"`), and users assigned to the role see only the matching rows. Test every role by viewing the report as that user, because a role that was never tested is a role that leaks.

The Python equivalent, useful for understanding the logic, filters the rows before they are returned:

Python

```python
import pandas as pd

sales = pd.read_csv("superstore_sales.csv", parse_dates=["order_date"])

def rows_for_user(df, user_region):
    # The filter is applied on the server side, before anything reaches the user
    return df[df["region"] == user_region]

north = rows_for_user(sales, "North")
print("rows visible to the North manager:", len(north), "| regions present:",
north["region"].unique())
```

OUTPUT

```
rows visible to the North manager: 1530 | regions present: ['North']
```

NOTE **Test the boundary**

Check what a user sees at the edges: a row with a blank region, a manager who covers two regions, a user with no role at all. A user with no role should see nothing, not everything.

Access control and audit

Access control answers "who may do what". Good practice:

- **Least privilege.** Analysts read the views; only a small group reads raw tables.
- **Groups, not individuals.** Grant access to a role, and manage membership in one place.
- **Review access quarterly.** Remove people who changed teams or left.
- **Audit everything sensitive.** Log who queried which sensitive table and when, and review unusual access, such as a large export at night.

Audit field	Example
Time	2026-11-11 22:14
User	analyst.ravi
Object	customers_raw
Action	SELECT, 4,200 rows
Purpose recorded	Ticket review

INTERVIEW CORNER

How would you let analysts work with customer data without exposing personal details?

Give them a view with personal fields removed or masked, grant raw access only to a named group with a documented purpose, apply row-level security where a manager should see only their area, and log access to the raw table. Test each role by viewing results as that role, and review access quarterly.

::::key What to remember - Removal is the strongest protection; masking and pseudonymisation keep more utility. - Use views to present safer versions of a table to analysts. - Use keyed pseudonyms, with the key stored separately; plain hashes are reversible in practice. - Row-level security filters rows by user; test every role, including edge cases. - Audit access to sensitive data and review it. ::::

Quick quiz

5 questions · tap an answer to check it

1. Which method keeps the most analytical utility while hiding identities?

A Removal

B Masking the whole value

C Pseudonymisation with a keyed token

D Deleting the table

Answer: C. Keyed tokens keep joins and counts while hiding the person.

2. Why is a plain hash of a phone number not enough?

A Hashes are too slow

B Anyone can hash every possible number and match the result

C Hashes are always reversible by law

D Hashes remove the data

Answer: B. Without a secret key, the space of inputs can be tested.

3. What does row-level security control?

A Which columns are visible

B Which rows each user can see

C How fast a query runs

D Where the data is stored

Answer: B. RLS filters rows by user.

4. A user has no role assigned. What should they see?

A Everything

B Nothing

C The last report

D The raw table

Answer: B. A missing role must fail closed.

5. Why keep audit logs for sensitive tables?

A They make queries faster

B So that unusual or unauthorised access can be detected and explained

C They replace access control

D They are required for charts

Answer: B. Audit logs support detection and accountability.

EXERCISE **Protect a support dataset**

Using the customers file, build a view that removes the full name and keeps identifiers and region, write a row-level rule so that a North manager sees only North customers, and list the audit fields you would log for a query on the raw table. Test the rule with a user who has no role and confirm they see nothing.

Show solution (worked answer)

Hints: the view can use the masking expression from this lesson. Test the no-role case by making the filter compare against an empty value, which should return no rows. Keep the audit list to fields a reviewer could use.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.